Lab Sheet 09

Title: ESP32 IoT System with MQTT and Node-RED Integration

Aims:

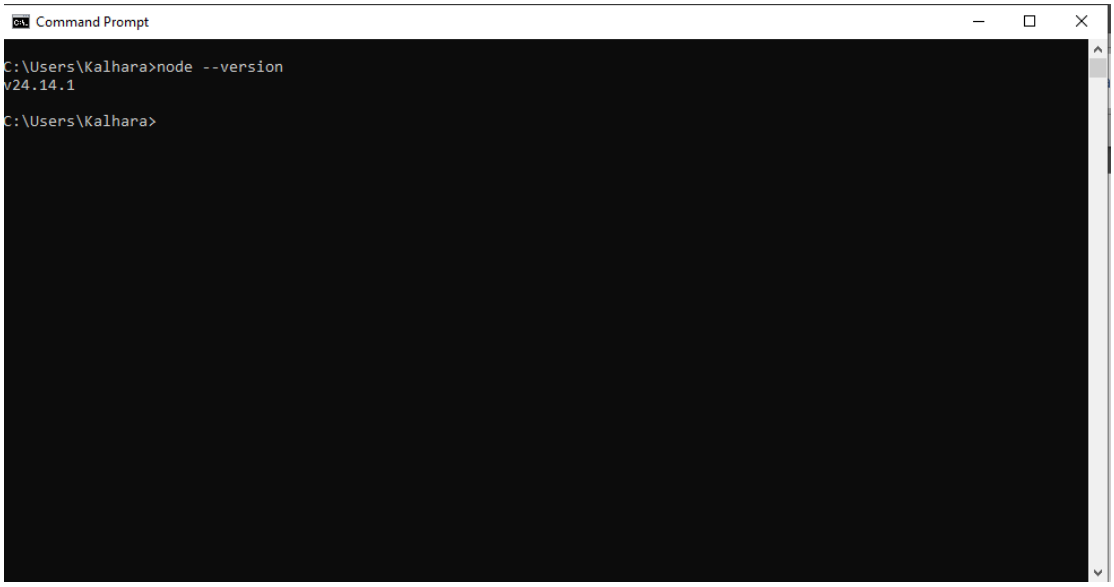
To design, simulate, and deploy an ESP32-based IoT system that publishes sensor data via MQTT and visualizes it using Node-RED dashboards. Learners will develop firmware, configure an MQTT broker, build Node-RED flows, and create real-time monitoring dashboards for a Smart Room Environmental Monitor.

Tasks: Smart Room Environmental Monitor

Part 1: Environment Setup

1. Install extensions in VS Code
    a. MQTT Explorer – standalone MQTT client for debugging
    b. PlatformIO IDE by PlatformIO – compile ESP32 firmware
    c. Node.js (LTS) – required for Node-RED runtime

node –version



d. Wokwi by Wokwi – browser-based ESP32 simulator

2. Install Node-RED globally via npm:

npm install -g --unsafe-perm node-red

```
C:\Users\Kalhara>node --version
v24.14.1

C:\Users\Kalhara>npm install -g --unsafe-perm node-red
npm warn "node-red" is being parsed as a normal command line argument.
npm warn Unknown cli config "--unsafe-perm". This will stop working in the next major version of npm.

added 5 packages, removed 62 packages, and changed 365 packages in 3m

70 packages are looking for funding
  run `npm fund` for details

C:\Users\Kalhara>
```

3. Install an MQTT Broker (Mosquitto):

a. Download from https://mosquitto.org/download/

b. After install, start the broker:

i. mosquitto -v

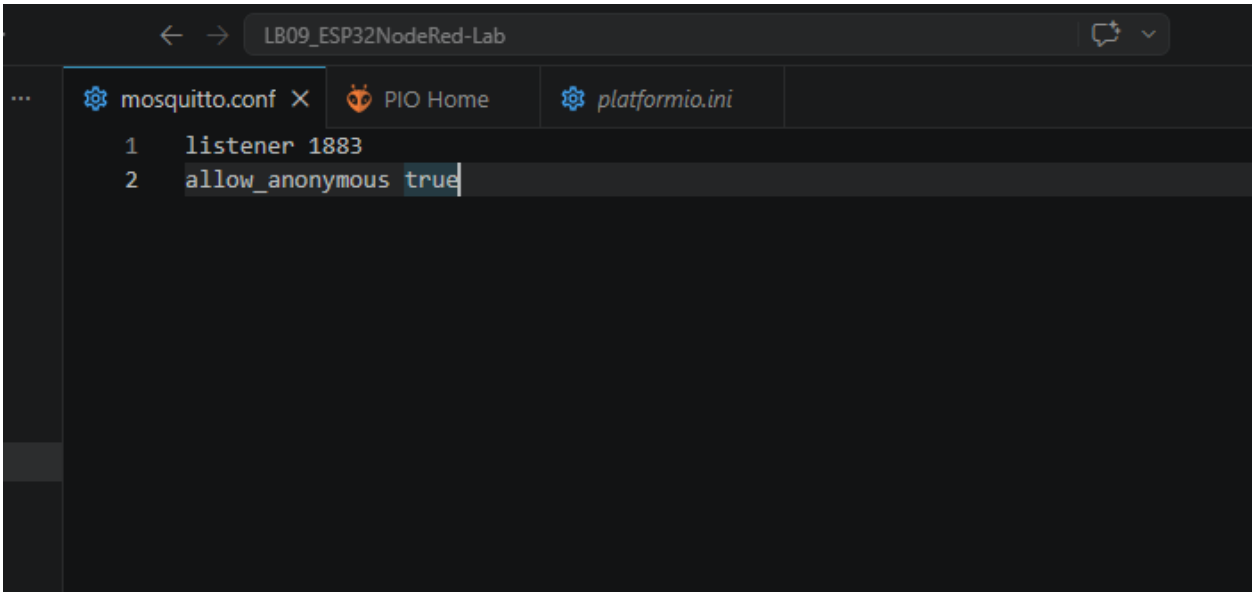
```
C:\Users\Kalhara>npm install -g --unsafe-perm node-red
npm warn "node-red" is being parsed as a normal command line argument.
npm warn Unknown cli config "--unsafe-perm". This will stop working in the next major version of npm.

added 5 packages, removed 62 packages, and changed 365 packages in 3m

70 packages are looking for funding
  run `npm fund` for details

C:\Users\Kalhara>mosquitto -v
1783178338: mosquitto version 2.1.2 starting
1783178338: Using default config.
1783178338: Bridge support available.
1783178338: Persistence support available.
1783178338: TLS support available.
1783178338: TLS-PSK support available.
1783178338: Websockets support available.
1783178338: Starting in local only mode. Connections will only be possible from clients running on this machine.
1783178338: Create a configuration file which defines a listener to allow remote access.
1783178338: For more details see https://mosquitto.org/documentation/authentication-methods/
1783178338: Opening ipv4 listen socket on port 1883.
1783178338: Error: Only one usage of each socket address (protocol/network address/port) is normally permitted.

1783178338: Opening ipv6 listen socket on port 1883.
1783178338: Error: Only one usage of each socket address (protocol/network address/port) is normally permitted.

1783178338: mosquitto version 2.1.2 terminating

C:\Users\Kalhara>
```

c. Create file named **mosquitto.conf** and save on working folder.

i. Type below codes:

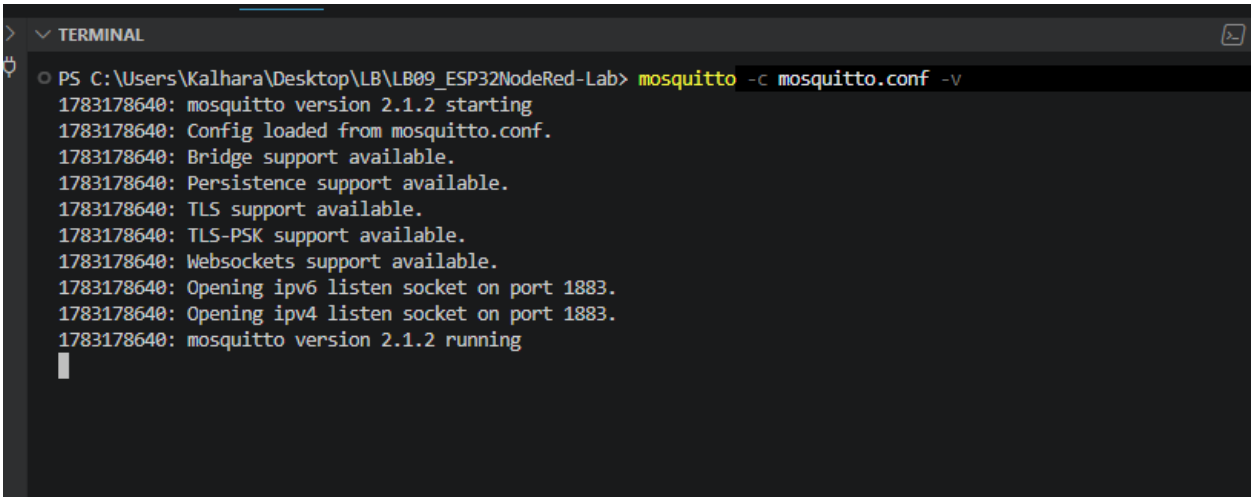
listener 1883

allow_anonymous true



d. Start the broker.

i. In cmd : *mosquitto -c mosquitto.conf -v*

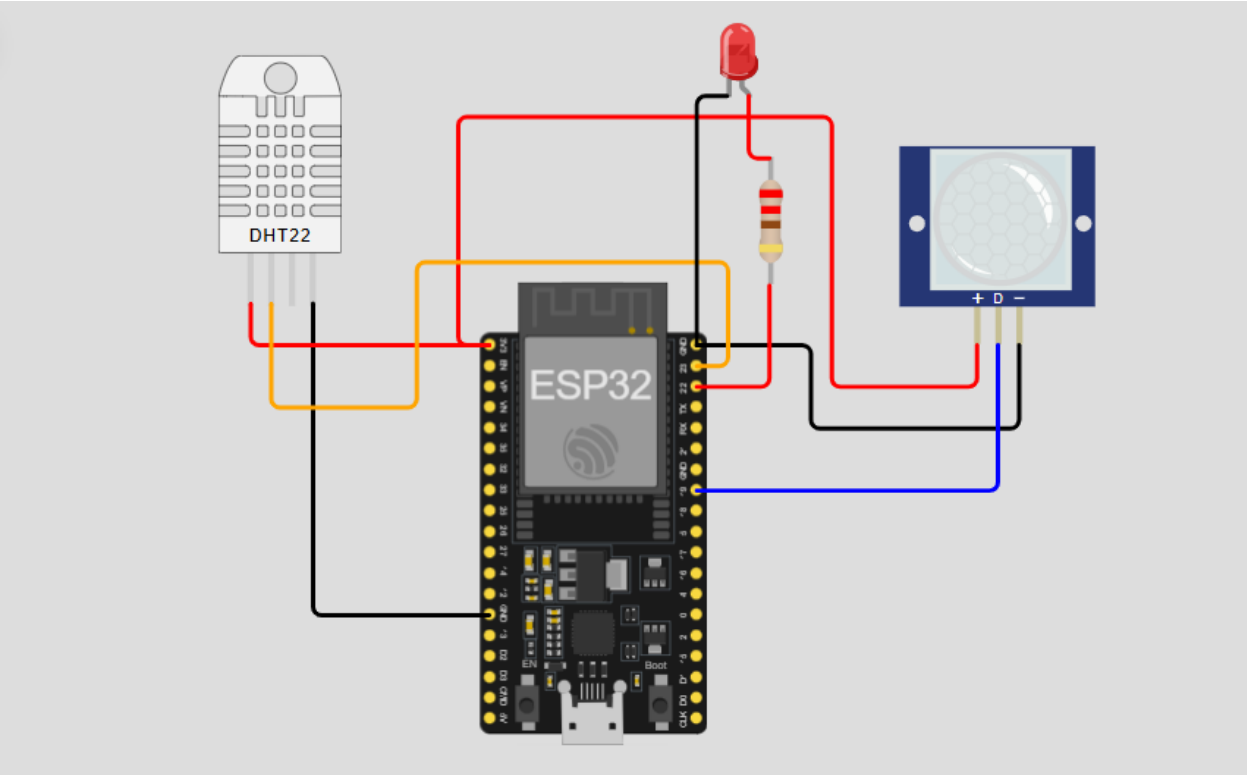


Part 2: Project Setup in Wokwi / VS Code

1. Name the project **ESP32NodeRed-Lab**.

2. Circuit Design

    a) Connect the DHT22 sensor to an ESP32 GPIO pin.
    b) Connect the PIR sensor to an ESP32 GPIO pin.
    c) Connect the LED to the ESP32.



3. Library Integration

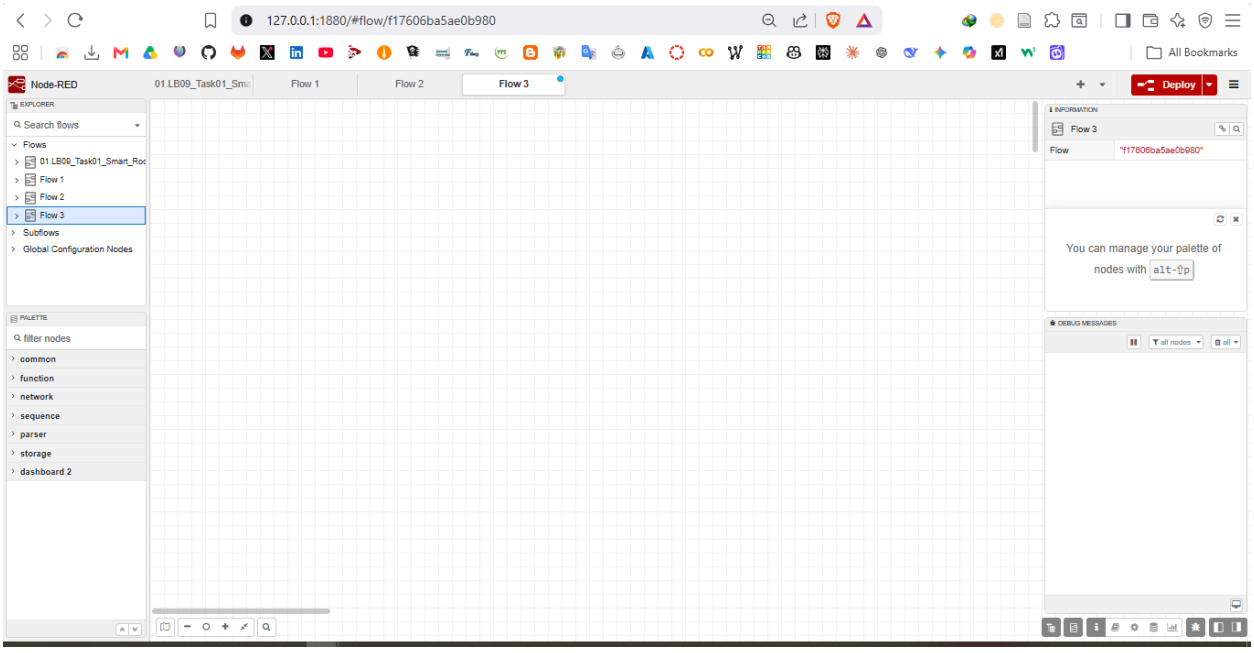
    a) PubSubClient
    b) DHT sensor library

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
lib_deps =
    adafruit/DHT sensor library@^1.4.7
    knolleary/PubSubClient@^2.8
```

4. Configure Node-RED

    a. Launch Node-RED

```
C:\Users\Kalhara>node-red
4 Jul 21:18:41 - [info]

Welcome to Node-RED
===================

4 Jul 21:18:41 - [info] Node-RED version: v5.0.1
4 Jul 21:18:41 - [info] Node.js  version: v24.14.1
4 Jul 21:18:41 - [info] Windows_NT 10.0.19045 x64 LE
4 Jul 21:18:43 - [info] Loading palette nodes
4 Jul 21:19:05 - [info] Settings file  : C:\Users\Kalhara\.node-red\settings.js
4 Jul 21:19:05 - [info] Context store  : 'default' [module=memory]
4 Jul 21:19:05 - [info] User directory : \Users\Kalhara\.node-red
4 Jul 21:19:05 - [warn] Projects disabled : editorTheme.projects.enabled=false
4 Jul 21:19:05 - [info] Flows file     : \Users\Kalhara\.node-red\flows.json
4 Jul 21:19:05 - [warn]

---------------------------------------------------------------------
Your flow credentials file is encrypted using a system-generated key.

If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
```



b. Install Required Nodes

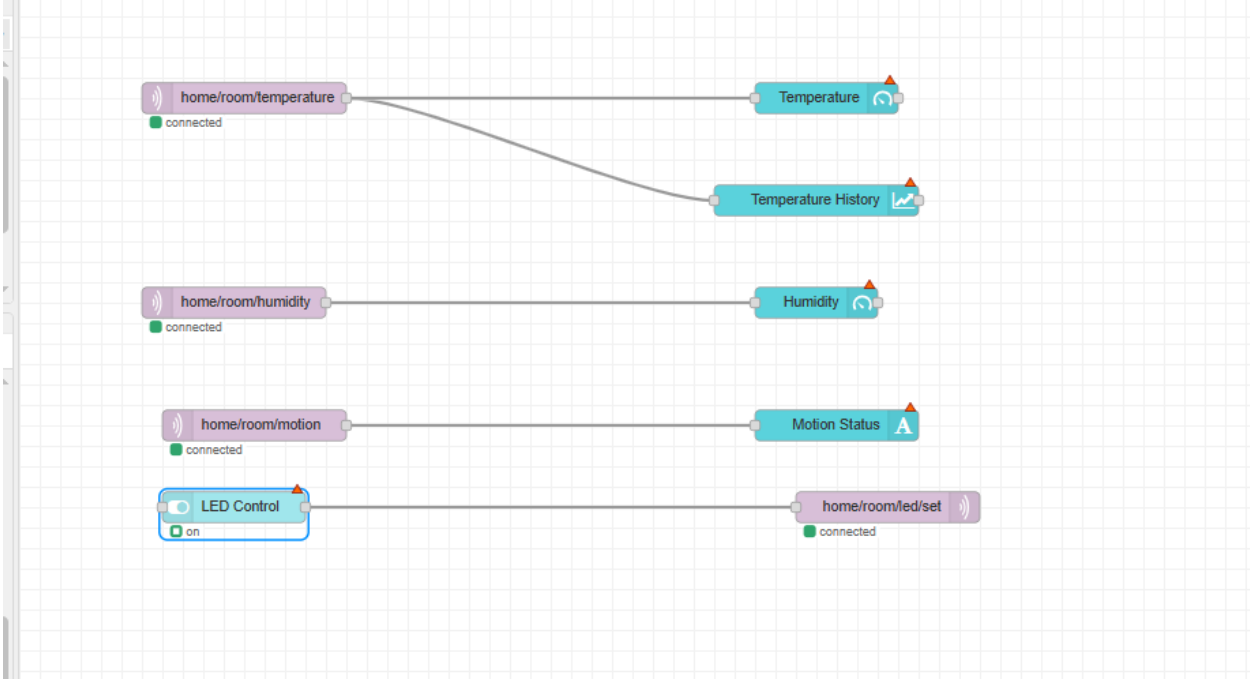
    i. @flowfuse/node-red-dashboard

c. Build the MQTT Subscriber Flow

Node	Type	Configuration
Temperature Flow		
mqtt in	Input	Topic: home/room/temperature, Server: ip_address:1883
gauge	Dashboard	Label: Temperature, Min: 0, Max: 80, Unit: °C
chart	Dashboard	Label: Temperature History
Humidity Flow		
mqtt in	Input	Topic: home/room/humidity Server: ip_address:1883
gauge	Dashboard	Label: Humidity, Min: 0, Max: 100, Unit: %
Motion Detection Flow		
mqtt in	Input	Topic: home/room/motion
text	Dashboard	Label: Motion Status
LED Control Flow		
switch	Dashboard	Label: LED Control, On Payload: ON, Off Payload: OFF
mqtt out	Output	Topic: home/room/led/set, Server: ip_address:1883

d. Click Deploy (top-right red button) after wiring all nodes.

e. Open the dashboard at http://localhost:1880/dashboard to view live data.



5. Programming Logic Development in Wokwi / VS Code

   a. Initialize Wi-Fi and MQTT connection to enable network communication between ESP32 and the broker.

   b. Read temperature and humidity values from the DHT22 sensor using the appropriate sensor library.

   c. Read and process PIR sensor data for motion detection.

   d. Publish all sensor readings to MQTT topics in real time for Node-RED dashboard visualization.

```
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

// Wi-Fi සහ MQTT Configurations
const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqtt_server = "192.168.116.232";

// Pin Definitions
```

```
#define DHTPIN 23
#define DHTTYPE DHT22
#define PIR_PIN 19
#define LED_PIN 22

// MQTT Topics
#define TOPIC_TEMP "home/room/temperature"
#define TOPIC_HUMID "home/room/humidity"
#define TOPIC_MOTION "home/room/motion"
#define TOPIC_LED_SET "home/room/led/set"

DHT dht(DHTPIN, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastMsg = 0;

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

// MQTT හරහා පණිවිඩ ලැබෙන විට ක්‍රියාත්මක වන ශ්‍රිතය (Callback)
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");

  String message = "";
  for (int i = 0; i < length; i++) {
```

```
    message += (char)payload[i];
  }
  Serial.println(message);

  // LED එක පාලනය කිරීම
  if (String(topic) == TOPIC_LED_SET) {
    if (message == "ON") {
      digitalWrite(LED_PIN, HIGH);
      Serial.println("LED turned ON");
    } else if (message == "OFF") {
      digitalWrite(LED_PIN, LOW);
      Serial.println("LED turned OFF");
    }
  }
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32Client-";
    clientId += String(random(0xffff), HEX);

    if (client.connect(clientId.c_str())) {
      Serial.println("connected");
      // LED Topic එක Subscribe කිරීම
      client.subscribe(TOPIC_LED_SET);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  dht.begin();
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}
```

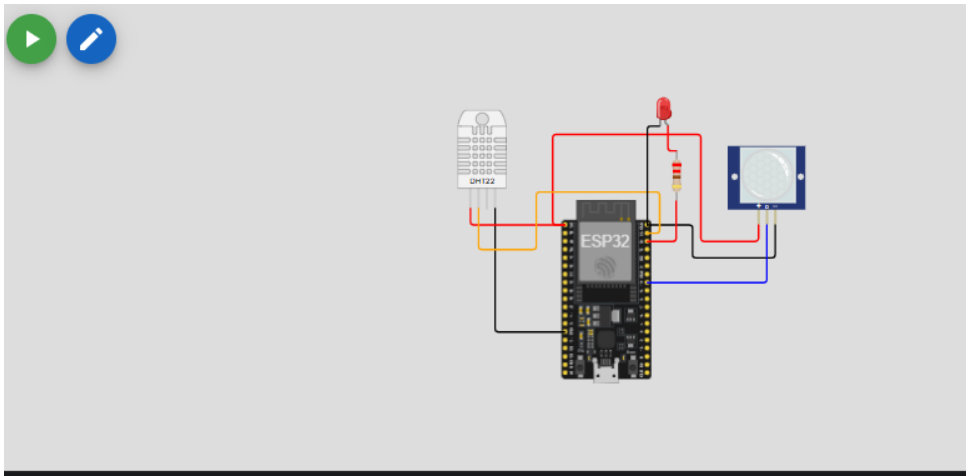
```
void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  unsigned long now = millis();

  if (now - lastMsg > 2000) {
    lastMsg = now;

    // DHT22 දත්ත කියවීම
    float h = dht.readHumidity();
    float t = dht.readTemperature();


    int motionDetected = digitalRead(PIR_PIN);

    if (isnan(h) || isnan(t)) {
      Serial.println("Failed to read from DHT sensor!");
      return;
    }


    Serial.print("Temp: "); Serial.print(t);
    Serial.print(" | Humid: "); Serial.print(h);
    Serial.print(" | Motion: "); Serial.println(motionDetected ? "MOTION!" : "No
Motion");


    client.publish(TOPIC_TEMP, String(t).c_str());
    client.publish(TOPIC_HUMID, String(h).c_str());
    client.publish(TOPIC_MOTION, motionDetected ? "Motion Detected" : "Clear");
  }
}
```

6. Simulation & Testing in Wokwi / VS Code

    a. Run simulation

    b. Adjust values in Wokwi / VS Code

    c. Verify Node-RED interaction



PROBLEMS OUTPUT TERMINAL

TERMINAL

```
Temp: 53.30 | Humid: 64.50 | Motion: No Motion
Temp: 53.30 | Humid: 64.50 | Motion: No Motion
Temp: 53.30 | Humid: 64.50 | Motion: No Motion
Temp: 53.30 | Humid: 64.50 | Motion: No Motion
Message arrived [home/room/led/set] ON
LED turned ON
Temp: -5.20 | Humid: 42.50 | Motion: MOTION!
Temp: -5.20 | Humid: 42.50 | Motion: MOTION!
Temp: -5.20 | Humid: 42.50 | Motion: No Motion
Temp: -5.20 | Humid: 42.50 | Motion: No Motion
```